

The qLDPC Challenge

A public, automatically verified leaderboard for quantum LDPC codes

Unitary Foundation

github.com/unitaryfoundation/qldpc-challenge

Extended abstract, lightning talk, Quantum Software Workshop 2026 (IEEE Quantum Week, Toronto)

Abstract

Quantum low-density parity-check (qLDPC) codes are the leading route to low-overhead fault tolerance, but the field has no shared, machine-checked record of which codes are best under which constraints. Parameters are scattered across papers and tables, and distance claims in particular are hard to reproduce and easy to overstate, since computing distance is NP-hard and is often estimated with heuristic decoders that can be wrong. The qLDPC Challenge is a public leaderboard that treats this as a software-infrastructure problem. A participant submits one JSON file describing a CSS code; continuous integration recomputes the code’s properties from its parity-check matrices, checks a self-certifying distance witness, and merges the entry only if it holds up. Ranking is a per-track Pareto frontier rather than a single gameable number, track membership is computed rather than self-declared, and the whole pipeline (validator, distance refutation gate, exact certifier, scorer, static-site generator) lives in the repository and runs on every pull request. We describe the trust model, the verification pipeline, and the current board, and argue the design is a reusable pattern for reproducible, self-certifying benchmarks in quantum error correction.

1 Motivation

Recent qLDPC constructions, from the bivariate-bicycle “gross” code $[[144, 12, 12]]$ of Bravyi et al. to planar open-boundary families, have made order-of-magnitude reductions in the qubit overhead of fault tolerance plausible. Progress is now fast and distributed across many groups, which makes a shared question pressing: for a given hardware constraint (check weight, geometric locality, code size), what is the best known code, and can we trust the claim?

Two facts make this hard to answer in the literature alone. First, results live in heterogeneous tables across papers, with no common format or single place to compare like with like. Second, and more seriously, the headline parameter, distance d , is the one that is hard to verify. Finding a low-weight logical operator gives only an upper bound on d ; proving $d \geq D$ is the NP-hard direction. Reported distances are frequently decoder-based upper bounds, and recent work documents belief-propagation decoders overestimating qLDPC distances by large factors. A leaderboard that simply trusts submitted numbers would accumulate quiet errors.

The qLDPC Challenge addresses both problems with quantum-software tooling rather than a static document. Every entry is a small machine-readable file that is automatically re-verified, distance claims must carry a checkable witness, and a distance refutation step actively searches for counterexamples. The result is a reproducible, community-extensible, self-certifying record.

2 Design

Three principles shape the system.

(i) Everything cheap is trustless and mandatory. The verifier recomputes n , the logical dimension $k = n - \text{rank}_{\mathbb{F}_2}(H_X) - \text{rank}_{\mathbb{F}_2}(H_Z)$, CSS commutation $H_X H_Z^\top = 0$ over $\mathbb{F}_2$, the maximum check weight, and the geometric locality class, all from the submitted parity checks. A wrong claimed k or a non-commuting “stabilizer” is rejected in milliseconds. Nothing in this tier is taken on trust.

(ii) Distance is layered by how much is proven. Because exact distance does not scale, the board separates three confidence tiers. An *upper bound* ($d \leq$) is backed by an explicit logical-operator witness that the verifier confirms is a genuine nontrivial logical of the claimed weight, so the bound is checkable with no trust required. A *corroborated* bound ($d \leq^*$) is an upper bound that an independent search has tried and failed to beat. An *exact* value ($d =$) is one a server-side integer program has proven minimal. Nothing is ever silently upgraded.

(iii) A code is not a single number. The primary tracks are a grid of two nested, *computed* axes: a locality class (`local-2d-single` $\subset$ `local-2d-bilayer` $\subset$ `unrestricted`, derived from the layout) and a check-weight class (`weight-4` $\subset$ `weight-6` $\subset$ `weight-8` $\subset$ any). Membership is derived from the parity checks and the layout, so a track cannot be claimed by relabeling. Within each cell the ranking is a Pareto frontier over (n, k, d, w) ; there is no single winner. The construction family (bivariate bicycle, generalized bicycle, two-block group algebra, tile, topological, ...) cannot be recovered from the matrices, so it is a filterable tag, never a ranking.

3 The verification pipeline

The pipeline is the reusable artifact. On every pull request, GitHub Actions runs, in order:

1. **Structural and trustless checks.** Schema validity, qubit indices in range, no repeated index within a check, CSS commutation, recomputed k matching the claim, $k \geq 1$, a controlled family and novelty vocabulary, and the witness check: each distance witness must have weight equal to the claimed value, lie in the kernel of the opposite checks, and be a nontrivial logical (not a stabilizer product). This certifies the per-side upper bound.
2. **Computed locality class.** If a layout is present, the verifier derives the locality class from the coordinates, enforcing no cramming (at most `layers` qubits per site, distinct sites at least unit-spaced) and a bounded interaction radius, so a small radius cannot be faked by collapsing qubits.
3. **Distance refutation gate.** Two independent searches, a random-information-set search and a syndrome-decoder search, look for a logical lighter than the claimed distance. Any hit is a sound refutation (it exhibits a checkable counterexample). The per-run seed is random by design so an over-claim cannot be tuned to evade one fixed search, and the gate fails closed. A weekly whole-board sweep re-runs the refutation with fresh seeds to accumulate coverage over time.
4. **Identity and duplicate checks.** The pull-request author must be a listed author of the codes they add. Every code is fingerprinted two ways: the reduced-row-echelon form of (H_X, H_Z) pins the exact stabilizer group (an identical fingerprint is a hard error), and a permutation-invariant Weisfeiler–Leman signature on the Tanner graph flags possible equivalents for review.

A separate, maintainer-run exact tier upgrades a claim to $d =$. For each logical generator t_L it asks, via a bounded integer program (scipy/HiGHS, no commercial solver), whether a v exists with $Hv \equiv 0$, $\langle v, t_L \rangle \equiv 1$, and $\text{wt}(v) \leq \text{value} - 1$. If every generator on both sides is infeasible, the distance is exact; if a lighter logical is found, the claim is refuted; if the solver times out, the side honestly remains an upper bound. A certificate written to `certs/` is what promotes the board label.

4 Metric and the state of the board

The figure of merit the 2D-locality literature uses, kd^2/n , is the Bravyi–Poulin–Terhal saturation ratio. For a 2D-local or bounded-weight code it is bounded by a constant that depends only on the interaction range, check weight, and on-site dimension, not on n , so maximizing it within a fixed locality model is well posed. For asymptotically good codes, where k and d both grow with n , it grows like n^2 without bound. The board therefore treats kd^2/n as a per-cell, sortable column and compares like with like, never as a global record. Table ?? lists reference points.

Code	$[[n, k, d]]$	topology	w	kd^2/n
Rotated surface (Kitaev)	$[[d^2, 1, d]]$	planar	4	1
Gross code (Bravyi et al.)	$[[144, 12, 12]]$	periodic	6	12
Tile code (Eberhardt et al.)	$[[512, 18, 19]]$	planar	8	12.7
Panteleev–Kalachev (GB)	$[[126, 28, 8]]$	periodic	10	14.2

Table 1: Reference points; kd^2/n values for 2D-local weight-8 codes are exact, integer-programming-certified.

At the time of writing the board holds 40 verified codes, 21 submitted through the challenge and 19 literature baselines, with 27 certified exact, across the bivariate-bicycle, generalized-bicycle, two-block group-algebra, tile, and topological families. It is seeded against the published state of the art (the planar codes of Liang, Eberhardt, and Chen; the bivariate-bicycle codes of Bravyi et al.; the generalized-bicycle codes of Panteleev–Kalachev and Wang–Pryadko; and the Kitaev and Steane baselines), so a submission is always measured against known codes. The seed set is selected, not an exhaustive snapshot, and the board states this: an on-board leader may still be matched by a published code that has not been seeded. The badges and figures are regenerated from the live board on every build, so they track the current numbers rather than a hand-typed count.

5 Relevance to quantum software, and the talk

The challenge is a small case study in software infrastructure for quantum error correction. It encodes a trust model directly in CI: cheap properties are recomputed and mandatory, distance is self-certifying at the cheap tier and separately earned at the exact tier, and an adversarial refutation step guards the one quantity that is expensive to prove. The submission unit is a verifiable file, the referee is a script anyone can run locally, and the contribution workflow is a pull request, which lowers the barrier to comparing codes while keeping over-claims out of the record. The verifier and the refutation gate are reusable beyond this board.

The lightning talk will demonstrate the end-to-end flow (submit, CI verifies, board updates), the computed layered scheme and its trust model, and how to contribute, including via LLM-guided search over code-generating programs. We will also raise the questions the board is built to surface:

whether planar weight-6 codes can close the gap to periodic records, how far the refutation gate and exact certification scale, and which families have unmined finite-size headroom.

Key references

- Bravyi, Cross, Gambetta, Maslov, Rall, Yoder, Nature **627**, 778 (2024), arXiv:2308.07915 (bivariate-bicycle / gross code).
- Liang, Eberhardt, Chen, arXiv:2504.08887 (planar open-boundary qLDPC).
- Eberhardt et al., tile codes, arXiv:2504.09171.
- Panteleev, Kalachev, Quantum **5**, 585 (2021), arXiv:1904.02703; Wang, Pryadko, arXiv:2203.17216 (generalized bicycle).
- Bravyi, Poulin, Terhal, Phys. Rev. Lett. **104**, 050503 (2010), arXiv:0909.5200 (the kd^2/n bound).